



**COMSATS University Islamabad,
Sahiwal Campus.**

Red-Light Green-Light

By

M Hasseb Asghar

CIIT/SP22-BCS-137/SWL

M Zaid Maqsood

CIIT/SP22-BCS-167/SWL

Supervisor

Ms. Maryam Mazhar

Bachelor of Science in Computer Science (2022-2026)



**COMSATS University Islamabad,
Sahiwal Campus**

Red-Light Green-Light

A Project Presented To

**COMSATS University Islamabad,
Sahiwal Campus**

In partial fulfillment

of the requirement for the degree of

Bachelor of Science in Computer Science (2022-2026)

By

M Hasseb Asghar

M Zaid Maqsood

CIIT/SP22-BCS-137/SWL

CIIT/SP22-BCS-167/SWL

DECLARATION

We hereby declare that this software, neither whole nor as a part, has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely based on our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other. We will stand by the consequences. No portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

**Muhammad
Haseeb Asghar**

**Muhammad
Zaid Maqsood**

CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS(CS) “**Red-Light Green-Light**” was developed by **M Haseeb Asghar** (CIIT/SP22-BCS-137/SWL), **M Zaid Maqsood** (CIIT/SP22-BCS-167/SWL), under the supervision of “**Ms. Maryam Mazhar**” and co supervisor “**Ms. Tahreem Saeed**” and that in their opinion; it is fully adequate, in scope and quality for the degree of Bachelors of Science in Computer Science.

Supervisor

Mam Maryam Mazhar

Lecturer

Department of Computer Science

COMSATS University Islamabad, Sahiwal Campus

External Examiner

Dr. Ghulam Ali

Assistant Professor

Department of Computer Science

University of Okara

Head of Department

Dr. Zafar Iqbal Roy

Assistant Professor

Department of Computer Science

COMSATS University Islamabad, Sahiwal Campus

Acknowledgment

All praise is to almighty Allah who bestowed upon us a minute portion of his boundless knowledge by which we were able to accomplish this challenging task.

We are greatly indebted to our project supervisor “**Ms. Maryam Mazhar**” and our co-supervisor “**Ms. Tahreem Saeed**”. Without their supervision, advice, and valuable guidance, the completion of this project would have been doubtful. We are deeply indebted to them for their encouragement and continual help during this work.

And we are also thankful to our parents and family who have been a constant source of encouragement for us and brought us the values of honesty & hard work.

Executive Summary

This project is a multiplayer game based on the well-known Red Light Green Light challenge from the Squid Game series. The aim of the project is to build a complete and working game system where multiple players can join, play together, and compete in real time. The game is developed using Unity and C#, and Mirror Networking is used to connect players over the internet. When the game starts, players run forward during Green Light and stop instantly during Red Light. If any player moves during Red Light, the system eliminates them automatically. The first player who safely reaches the finish line becomes the winner.

To make the system more modern and secure, the project also includes PlayFab and Azure Functions for storing player data and match results on the cloud. A Solidity smart contract is added to record the winner's information on the blockchain. This gives transparency and ensures that the match results cannot be changed or tampered with. The project demonstrates how traditional online games can combine cloud computing and blockchain to create a fairer and more trustworthy gaming experience.

The project followed an Agile / Incremental development method, where each part of the system gameplay, networking, backend, and smart contract was built step by step and tested along the way. Various tests were performed to check movement, timing, elimination rules, multiplayer synchronization, and the saving of match results. The outcome is a functional multiplayer prototype that connects game mechanics, networking, cloud services, and blockchain in one complete system. The project also provides a foundation for future expansion such as mobile support, more Squid Game levels, and improved graphics or security.

Abbreviations

OOP	Object Oriented Programming
SDLC	Software Development Lifecycle
UI	User Interface

Table of Contents

1. Introduction.....	1
1.1 Brief Overview.....	1
1.2 Relevance to Course Modules	2
1.3 Project Background.....	3
1.4 Literature Review.....	3
1.5 Analysis from Literature Review.....	4
1.5.1 Limitations of Existing Multiplayer Systems	4
1.5.2 Gaps in Current Web3 Gaming Solutions	5
1.5.3 Opportunities Identified for Integration.....	5
1.6 Methodology and Software Lifecycle.....	5
1.6.1 Rational Behind Selected Agile Model.....	5
1.7 Summary	6
2 Problem Definition.....	9
2.1 Problem Statement	9
2.2 Deliverable and Development Requirements	9
2.3 Current System Limitations	10
2.4 Summary	10
3 Requirement Analysis.....	13
3.1 Use Case Diagrams	13
3.1.1 User Profile	13
3.1.2 Administrator	14
3.1.3 User Profile	15
3.1.4 Administrator	16
3.2 Functional Requirements	17

3.2.1	User Registration & Authentication.....	17
3.2.2	Game Mechanics.....	18
3.2.3	Smart Contract Integration.....	18
3.2.4	Multiplayer Gameplay	18
3.2.5	Single Player Mode.....	18
3.2.6	Multiplayer Mode	18
3.2.7	In-App purchases	19
3.2.8	Manage Users.....	19
3.3	Nonfunctional Requirements	19
3.3.1	Performance	19
3.3.2	Security	19
3.3.3	Scalability	20
3.3.4	Ease of Use	20
4	Design and Architecture.....	22
4.1	System Architecture.....	22
4.2	Data Representation	23
4.3	Process Flow	24
4.4	Sequence Diagram	25
4.5	Class Diagram.....	27
4.5.1	Components	28
4.5.2	Relationships.....	29
5	Implementation	32
5.1	Algorithm.....	32
5.1.1	LightSwitch Algorithm	32
5.1.2	Movement Detection Algorithm.....	32

5.1.3	Elimination Algorithm.....	32
5.1.4	Mirror Networking.....	33
5.1.5	Azure Functions.....	33
5.1.6	Smart Contract (Solidity).....	33
5.1.7	PlayFab Integration.....	33
5.2	User Interface.....	33
6	Testing and Evaluation.....	40
6.1	Manual Testing.....	40
6.1.1	System Testing.....	40
6.1.2	Unit Testing.....	41
6.1.3	Functional Testing.....	42
6.1.4	Integration Testing.....	44
6.2	Tools and Technologies.....	45
6.3	Summary.....	46
7	Conclusion and Future Work.....	48
7.1	Conclusion.....	48
7.2	Future Work.....	49
7.2.1	Mobile Applications (Android & iOS).....	49
7.2.2	Advanced AI Features.....	49
7.2.3	Enhanced Character Customization.....	49
7.2.4	Strong Security and Privacy Measures.....	49
7.2.5	Performance and Scalability Optimization.....	49
8	References.....	50

List of Figures

Figure 3.1 User Profile Use Case Diagram.....	14
Figure 3.2 Administrator Profile Use Case Diagram.....	15
Figure 4.1 Red-Light Green-Light Architecture	22
Figure 4.2 Red-Light Green-Light Process Flow	25
Figure 4.3 Red-Light Green-Light Sequence Diagram.....	26
Figure 5.1 Red-Light Green-Light Home Page	34
Figure 5.2 Gameplay.....	35
Figure 5.3 Waiting Lobby.....	36
Figure 5.4 PlayFab Studio.....	37
Figure 5.5 Withdraw Panel	38

List of Tables

Table 1-1 Literature Review	4
Table 2-1 Current System Limitations.....	10
Table 3-1 User Profile.....	15
Table 3-2 Administrator Profile.....	16
Table 6-1 Unit Testing: Authentication & User Management.....	41
Table 6-2.2 Functional Testing	43
Table 6-3 Integration Testing.....	44
Table 6-4 Tools and Technologies.....	45

Chapter 1

Introduction

1. Introduction

This project presents a multiplayer game inspired by the popular "Red Light, Green Light" challenge from the Squid Game series. Developed in Unity, the game combines real-time multiplayer networking with Web3 technologies [13] to create a transparent, secure, and competitive environment. The game mechanics require players to advance during the "Green Light" phase and stop immediately during the "Red Light" phase, with failure resulting in elimination. The gameplay is simple but demands precision, attention, and timing from the players. To ensure decentralization and fairness, the project integrates Solidity-based smart contracts deployed on a blockchain to manage reward distribution. It uses PlayFab for cloud-based player management and authentication, while Azure Functions handle serverless backend logic such as game result validation and leaderboard updates. Mirror Networking is used for real-time multiplayer support within Unity, allowing smooth and scalable communication between players and the game server.

The project aims to explore the convergence of Web3 and traditional multiplayer game development, offering players a fair, engaging, and rewarding experience. It also serves as a case study for incorporating decentralized technologies into mainstream game design.

1.1 Brief Overview

This project focuses on creating a multiplayer version of the "Red Light, Green Light" game from the Squid Game series. The game is built in Unity and allows multiple players to connect, move, and compete in real time. Players must run during the Green Light phase and stop immediately during the Red Light phase, or they are eliminated. Although the gameplay rules are simple, the challenge requires quick reactions and careful timing.

To make the system more secure and transparent, the project integrates Web3 technology. A Solidity smart contract is used to store and manage game rewards on the blockchain, ensuring fairness and preventing any kind of tampering. PlayFab is used for cloud-based user accounts and game data, and Azure Functions handle backend tasks like validating game results and updating the leaderboard. Mirror Networking powers the multiplayer system, allowing players to join the same match and communicate smoothly within the game.

Overall, the project brings together multiplayer gaming and decentralized technology to create a fair, engaging, and modern gaming experience. It also shows how blockchain can be used in traditional game development.

1.2 Relevance to Course Modules

The development of Red-Light Green-Light directly applies and integrates various concepts and skills acquired throughout our Computer Science course.

i. Object-Oriented Programming (OOP):

The game is developed in Unity using C#, where all major components like the player, game manager, doll controller, and networking scripts follow OOP principles such as classes, objects, inheritance, and encapsulation. Game mechanics like movement, state control (Red/Green), and elimination are implemented using OOP design patterns.

ii. Computer Networks:

Mirror Networking is used to build a real-time multiplayer environment. Concepts like client-server architecture, synchronization, latency handling, and network events directly relate to networking courses. The project demonstrates how data is shared across multiple clients in a distributed system.

iii. Database Systems:

PlayFab [7] is used for user account management, player data, statistics, and cloud storage. Azure Functions act as a serverless backend, which relates to cloud computing, REST APIs, and backend development taught in these modules.

iv. Web Technologies:

The project integrates Solidity smart contracts and Web3 [13] concepts. It applies what is learned about decentralized systems, hashing, immutability, and blockchain networks. Understanding how smart contracts work and how transactions are executed ties directly to modern Web and blockchain modules.

v. Software Engineering Concepts (SDLC):

The project follows an Incremental/Agile development approach. Concepts like requirement gathering, use case diagrams, testing, version control, and modular development are applied.

vi. Human-Computer Interaction (HCI):

The UI screens of the game (menus, lobby, gameplay UI, results screen) follow basic HCI principles such as simplicity, feedback, visibility, and easy navigation. Player experience and game controls are designed while considering usability.

1.3 Project Background

The Red-Light Green-Light game was conceived in October 2024 at COMSATS University Islamabad, Sahiwal Campus, under the supervision of Mam Maryam Mazhar, as part of the Bachelor of Science in Computer Science (2022–2026) final year project [1]. The idea for this project began at a time when blockchain technology, online multiplayer games, and cloud-based services were becoming increasingly common in modern software development. Many game developers and companies were experimenting with ways to combine traditional gameplay with decentralized systems to ensure fairness, transparency, and player trust. At the same time, Unity and Web3 tools were becoming easier to integrate, making it possible for students to create advanced game prototypes that use both multiplayer networking and blockchain features.

The “Red Light, Green Light” concept from the Squid Game series was selected because it offers simple rules but creates strong competition and excitement among players. It also provides a clear structure for testing real-time movement, timing, and synchronization in a multiplayer environment. Since the game depends heavily on precision and fairness, it was a suitable choice for integrating blockchain and cloud services to validate results and prevent tampering.

1.4 Literature Review

The development of multiplayer games has grown rapidly over the years, with real-time online games becoming a major part of the gaming industry. Many existing games rely on centralized servers to manage player data, match results, and reward distribution. Although this model works, it has limitations such as the risk of data manipulation, lack of transparency, and complete dependency on a single authority. Research in game development shows that players often demand fairness, competitive balance, and reliable systems that ensure equal opportunities for everyone. On the other hand, blockchain technology has gained attention for offering decentralization, immutability, and secure data storage. Studies highlight that smart contracts can be used in gaming to record outcomes in a transparent and tamper-proof way.

Table 1.1 Literature Review

Application Name	Weakness	Proposed Project Solution
Basic Motion Games	Limited accuracy in motion detection and elimination logic.	Our system ensures precise real-time motion detection and elimination.
Survival-Based Challenges	Lack engaging visuals and smooth controls, reducing player experience.	Incorporates the high-quality visuals and intuitive controls for immersion.

1.5 Analysis from Literature Review

The literature shows that while multiplayer games have advanced significantly, most still rely on centralized systems, which creates concerns about fairness and transparency. Blockchain technology, on the other hand, provides strong solutions for secure and tamper-proof data storage, but it is rarely combined with fast-paced, real-time gameplay. This gap indicates an opportunity to explore how decentralized systems can support competitive multiplayer environments.

The reviewed studies and existing technologies also suggest that Unity, Mirror Networking, PlayFab, and Azure Functions are well-suited for building scalable and interactive multiplayer systems [1]. These tools can be combined with smart contracts to record game outcomes in a transparent way that players can verify independently. Based on this analysis, the project aims to merge the strengths of traditional multiplayer development with decentralized technologies to address fairness and trust issues in online gaming.

1.5.1 Limitations of Existing Multiplayer Systems

Most real-time multiplayer games focus on gameplay mechanics and user engagement but do not provide transparent or verifiable match outcomes. Centralized storage also makes systems vulnerable to manipulation or data loss. Studies emphasize that players increasingly value fair and trustworthy systems, especially in competitive games.

1.5.2 Gaps in Current Web3 Gaming Solutions

Existing blockchain-based games rarely support fast-paced multiplayer interactions due to performance constraints and complex integration requirements. Most focus on NFTs, asset ownership, or turn-based mechanics, leaving a gap for interactive, skill-based gameplay that uses blockchain for transparent result recording rather than asset trading.

1.5.3 Opportunities Identified for Integration

Research also shows that Unity, Mirror Networking, PlayFab, and Azure Functions provide strong foundations for building reliable real-time multiplayer environments. Combining these tools with Solidity smart contracts offers a unique opportunity to create a more secure and tamper-proof system for storing match results [2]. This supports the idea of a game that blends traditional multiplayer development with decentralized technologies.

1.6 Methodology and Software Lifecycle

Red-Light Green-Light is a game project with diverse and evolving requirements, making it important to use a development methodology that supports flexibility, collaboration, and continuous refinement. Agile is an ideal fit because it allows the system to grow iteratively while adapting to user needs across students, researchers, and recruiters.

1.6.1 Rational Behind Selected Agile Model

Agile was chosen for this project because the system involves multiple complex components that must work together smoothly, such as real-time multiplayer networking, cloud-based services, and blockchain integration. These technologies require frequent testing, adjustments, and improvements, which makes a flexible development approach more practical than a rigid, fixed plan. Agile supports this by allowing the team to make changes quickly whenever a new issue appears, especially in areas like synchronization, response delays, or smart contract communication.

Another reason for choosing Agile is that game development naturally relies on continuous testing and refinement. Features like movement accuracy, player elimination, server authority, and Red Light/Green Light timing must be adjusted many times before they work correctly in a multiplayer environment. The Agile model supports this process by encouraging short development cycles, ongoing feedback, and regular improvements. This ensures that the project evolves step by step,

reduces risk, and produces a final system that is stable, responsive, and aligned with real gameplay needs.

Agile is based on the following principles:

1. **Iterative Development:** Core features such as player movement, Red Light/Green Light mechanics, multiplayer synchronization, backend communication, and smart contract interaction were built in small cycles. Each iteration improved the stability, performance, and accuracy of the gameplay.
2. **Incremental Delivery:** Game modules were released in usable parts, starting with a simple single-player prototype, then adding multiplayer networking, then integrating cloud services, and finally connecting blockchain functionality. This step-by-step approach ensured that each component worked correctly before the next one was added.
3. **Continuous Feedback:** Throughout development, testing was performed regularly to identify issues with timing, movement detection, multiplayer lag, and contract responses. Feedback from these tests guided improvements in gameplay flow, UI, and backend logic.
4. **Adaptive Planning:** The development plan was updated whenever new challenges appeared, such as multiplayer synchronization issues or changes in smart contract behavior. This allowed the project to stay aligned with real gameplay requirements and ensured smooth integration between Unity, PlayFab, Azure, and blockchain technologies.

1.7 Summary

The Red-Light Green-Light, started in October 2024 at COMSATS University Islamabad, Sahiwal Campus [1], was initiated as a modern approach to explore the combination of real-time multiplayer gameplay, blockchain technology, and cloud-based systems. Inspired by the iconic challenge from the Squid Game series, the project addresses the need for fairness, transparency, and trust in competitive online games. Traditional multiplayer games rely on centralized servers where match results and rewards can potentially be altered or mismanaged. Existing Web3 games, on the other hand, often lack real-time action and focus mainly on simple mechanics or asset ownership. This project bridges these gaps by offering real-time multiplayer gameplay, server-authoritative logic, and decentralized reward recording through blockchain smart contracts.

Developed using Unity and Mirror Networking, the system ensures smooth communication between players, accurate timing, and synchronized game states. PlayFab is used for player

authentication and cloud data management, while Azure Functions support serverless backend operations such as validating match results and updating leaderboards. A Solidity smart contract manages on-chain reward storage, ensuring tamper-proof and transparent outcomes. Built using the Agile methodology, the project follows iterative development, incremental updates, continuous testing, and adaptive planning. By combining Web3, cloud computing, and multiplayer game design, the project provides a forward-looking example of how decentralized technologies can enhance fairness, security, and trust in competitive gaming environments.

Chapter 2

Problem Definition

2 Problem Definition

This chapter discusses the precise problem to be solved and the intended outcome.

2.1 Problem Statement

Traditional multiplayer games rely on centralized servers to manage match outcomes, player progress, and reward distribution. This creates issues of fairness, transparency, and trust, as players must depend entirely on the game server to provide accurate and unaltered results. Existing Web3 games, on the other hand, offer decentralization [3] but lack fast, real-time gameplay and often do not support competitive action-based mechanics. There is a clear need for a system that combines real-time multiplayer gameplay with secure and tamper-proof result storage.

This project aims to address this problem by developing a Red-Light Green-Light multiplayer game that integrates Unity-based [4] real-time networking with blockchain technology and cloud services. The objective is to create a system where gameplay is synchronized, results are verifiable, and player rewards are recorded transparently through smart contracts. This ensures a fair, secure, and trustworthy competitive gaming experience.

2.2 Deliverable and Development Requirements

This project includes several key deliverables that support the complete development of a real-time multiplayer gaming system integrated with cloud and blockchain technologies [5]. The main deliverable is a functional Red Light Green Light multiplayer game developed in Unity, where players compete in real time using Mirror Networking. Additional deliverables include backend communication through PlayFab for player authentication and Azure Functions for validating match outcomes and updating leaderboards. A Solidity smart contract deployed on the blockchain is also delivered for storing match results transparently and ensuring tamper-proof reward distribution.

The development requirements include Unity and C# for game creation, Mirror for multiplayer networking, and PlayFab services for user management. Azure Functions are required to handle serverless logic, while Remix or Hardhat is used to develop and deploy the Solidity smart contract. Functional requirements include smooth multiplayer synchronization, elimination detection, Red Light Green Light state management, and secure result recording. Non-functional requirements

include low latency, system reliability, accurate state synchronization, and transparent storage of match outcomes on the blockchain.

2.3 Current System Limitations

Currently, most multiplayer games rely on centralized servers to control match outcomes, player progress, and reward distribution. This centralized structure makes systems vulnerable to manipulation, data loss, and lack of transparency. Players have no independent way to verify if the results were stored correctly or if the reward distribution was handled fairly. Existing gaming platforms also depend on traditional databases, which may not guarantee immutability or tamper-proof storage.

Web3 gaming solutions address decentralization but often fall short in supporting real-time, fast-paced gameplay. Many blockchain-based games remain turn-based or static due to performance constraints and complex integration processes. These limitations create a gap between traditional multiplayer games, which offer smooth gameplay but lack transparency, and decentralized games, which provide secure storage but do not offer dynamic multiplayer experiences. This project aims to address these limitations by merging real-time gameplay with transparent, blockchain-based result recording [6].

The following table summarizes the limitations found in existing games, highlighting the need for a more integrated, intelligent solution like Red-Light Green-Light:

Table 2.1 Current System Limitations

Platform	Limitations
Doodle Jump	Limited accuracy in motion detection and elimination logic.
Once Human	Lack engaging visuals and smooth controls, reducing player experience.

2.4 Summary

The analysis of existing systems shows clear gaps in fairness, transparency, and real-time performance within both traditional multiplayer games and current Web3 solutions. Centralized platforms lack verifiable result storage, while decentralized games often do not provide fast, interactive gameplay. With the advancement of cloud services and blockchain technologies, there

is a strong opportunity to integrate these features to create a more trusted and competitive gaming experience.

This project leverages Unity, Mirror Networking, PlayFab [7], Azure Functions, and smart contracts to build a complete multiplayer system that ensures secure gameplay and transparent result recording. The summary of this chapter highlights the need for a hybrid solution that combines the strengths of both centralized performance and decentralized trust. The proposed Red Light Green Light multiplayer game serves as a practical example of addressing these challenges through modern, integrated technologies.

Chapter 3

Requirement Analysis

3 Requirement Analysis

Requirement analysis is the process of identifying, analyzing, documenting, and managing stakeholder needs for a project or system to ensure the final product meets expectations, preventing costly rework by establishing a clear foundation and resolving ambiguities early on. It's a crucial phase in software and systems engineering, defining what a project should achieve, encompassing functional, technical, and business needs, and ensuring requirements are testable, actionable, and clear before development begins. This chapter discusses the requirements needed for the intended outcome.

3.1 Use Case Diagrams

In this Use case diagrams, we show the behavior of our system. This enables you to visualize the different types of roles in a system and how those roles interact with the system [5].

3.1.1 User Profile

This diagram represents the various interactions available to a User within the system. The user is connected to a wide range of actions, beginning with basic account operations such as logging in or registering. The diagram also shows wallet connectivity, which allows users to link their crypto wallet to the platform. Users can perform blockchain-related transactions, including depositing cryptocurrency into their account and withdrawing crypto back to their personal wallet. In addition to financial actions, users can engage with gameplay features like playing in single-player mode or participating in multiplayer mode. The diagram further highlights in-app purchases and in-game shop purchases, showing that users can buy digital items, upgrades, or assets either through the application interface or within the in-game shop. Collectively, this diagram demonstrates the full gameplay and crypto-transaction ecosystem accessible to regular users.

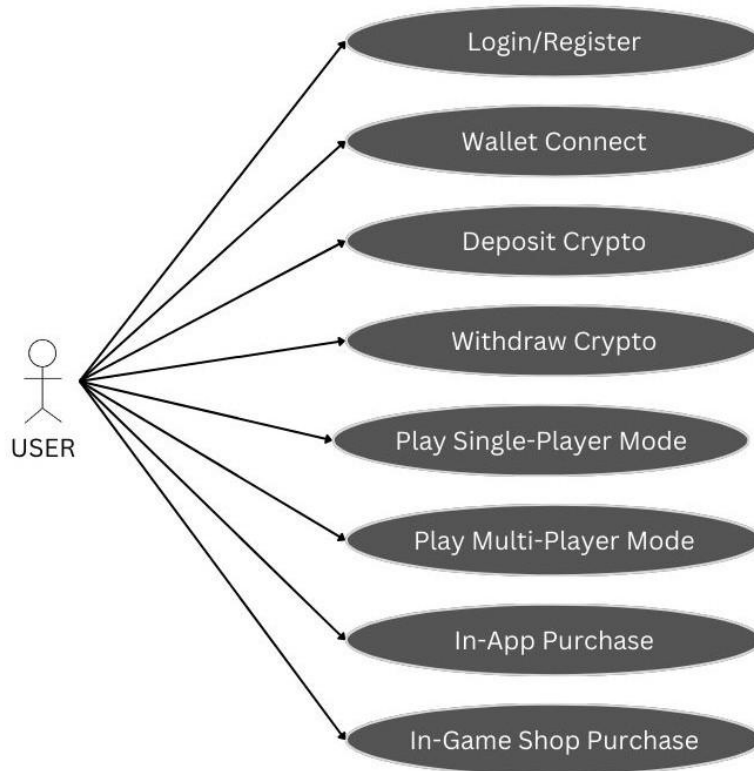


Figure 3.1 User Profile Use Case Diagram

3.1.2 Administrator

This diagram illustrates the main system interactions available to the Administrator. The administrator, represented by a stick-figure actor, is connected to several system-level operations. These include managing users, which involves creating, updating, or removing accounts and controlling user permissions. The diagram also shows that the administrator has the authority to disable queues, allowing them to halt or adjust matchmaking or service queues when necessary. Another important function is managing in-game assets, meaning the administrator can oversee the digital items, currencies, or resources used throughout the game environment. Finally, the administrator can update the smart contract, giving them control over blockchain operations such as deploying new contract versions or modifying existing ones. Overall, the diagram presents the administrator as having high-level access to the platform's operational and blockchain components.

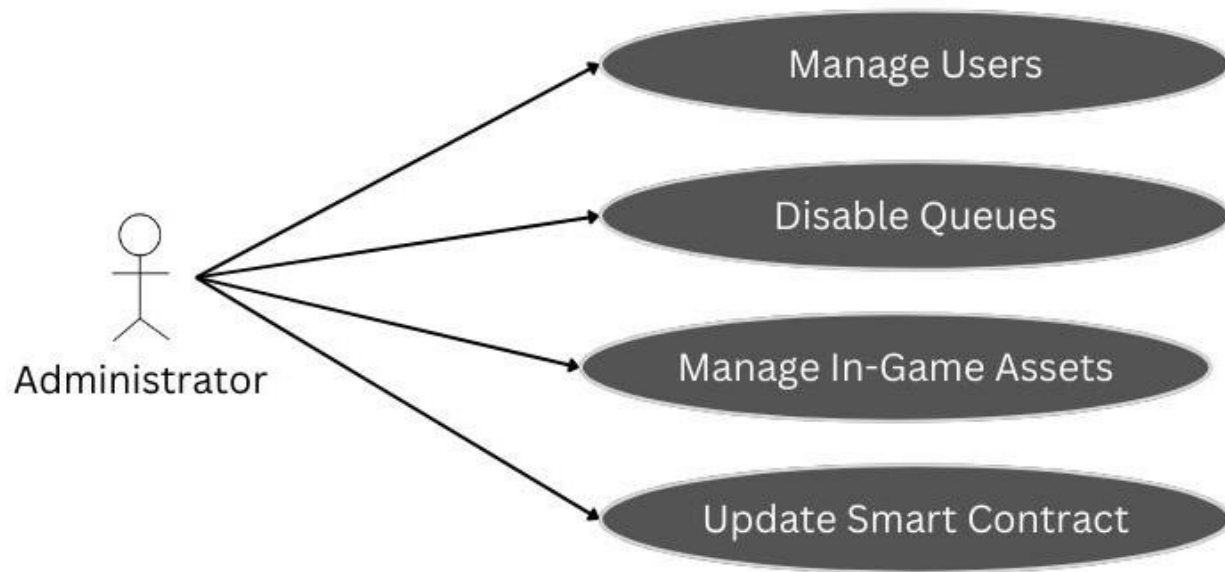


Figure 3.2 Administrator Profile Use Case Diagram

1.1.1.1 Use Case Description

3.1.3 User Profile

Table 3.1 defines how user interacts with the platform for gameplay and blockchain-related activities. They begin by registering or logging in and can connect their crypto wallet to enable transactions. Users can deposit or withdraw cryptocurrency and access both single-player and multiplayer game modes. They also make in-app or in-game shop purchases to enhance their gaming experience. This diagram highlights the full range of actions available to a regular player.

Table 3.1 User Profile

Use Case ID:	UC-01
Use Case Name:	User Profile
Actors:	Primary: User
Description:	The user participates in the game, interacts with the system, and competes in the "Red Light Green Light" challenge.

Trigger:	User registers, updates their account, or customizes their portfolio.
Preconditions:	The user must be authenticated and have an active account.
Postconditions:	The user either wins or loses the game, and the results are recorded on the blockchain.
Normal Flow:	1. The user logs into the game via PlayFab authentication. 2. The user connects their Web3 wallet. 3. The user joins a game lobby. 4. The user follows the "Red Light Green Light" rules. 5. If successful, the user wins rewards via smart contracts. 6. If the user fails, they are eliminated from the game.
Alternative Flows:	1.If authentication fails, the user cannot proceed. 2.If the wallet is not connected, the user cannot stake and participate. 3.If network issues occur, the user may disconnect.

3.1.4 Administrator

Table 3.2 explains the interactions of administrator with the system to perform essential management and control operations. They can manage users by adding, removing, or updating account information. The administrator also can disable queues when needed to ensure smooth system functioning. Additionally, they oversee in-game assets and maintain the smart contract used for blockchain operations. Overall, the admin ensures stability, security, and proper functioning of the platform.

Table 3.2 Administrator Profile

Use Case ID:	UC-02
Use Case Name:	Administrator
Actors:	Primary: Administrator
Description:	The administrator manages users, game settings, player accounts, in-game assets prices and system configurations.

Preconditions:	The administrator must be logged into the admin panel with the necessary privileges.
Postconditions:	The administrator ensures the game runs smoothly, and system updates are implemented as needed.
Normal Flow:	1. The administrator logs into the system. 2. The administrator monitors active games. 3. The administrator configures game parameters (e.g., reward structure, game rules). 4. The administrator handles player reports and disputes. 5. The administrator updates the smart contract if required.
Alternative Flows:	If authentication fails, the administrator cannot proceed.
Assumptions:	Data sources remain updated.

3.2 Functional Requirements

Functional requirements are essential in software development because they clearly specify what the system must do to fulfill user's needs and expectations. These requirements define the exact features, operations, and behaviors the system should provide—such as portfolio creation, data processing, AI-based recommendations, or recruiter search tools. Their importance lies in establishing a shared understanding among developers, users, and stakeholders, ensuring that the system's capabilities are fully aligned with project goals. By identifying these functions early, functional requirements help avoid misunderstandings, minimize costly revisions, improve planning accuracy, and support thorough testing and validation [11]. Ultimately, they guide the development process and ensure that the final product delivers meaningful value to its users.

These requirements include:

3.2.1 User Registration & Authentication

The system includes user registration and authentication through PlayFab, allowing players to securely log in before starting the game. Wallet connectivity is also required so users can participate in Web3-related actions, such as entering matches or receiving blockchain-based

rewards. These authentication mechanisms ensure that each player's identity and actions are securely verified within the platform.

3.2.2 Game Mechanics

The game mechanics follow the classic Red Light Green Light rules, where players must move during the green phase and stop immediately during the red phase. Any movement during the restricted phase results in automatic elimination, handled through precise detection logic. This ensures fair gameplay and maintains the competitive nature of the original concept. Additional mechanics ensure smooth transitions between phases and accurate player state tracking.

3.2.3 Smart Contract Integration

Smart contract integration enables cryptocurrency staking and reward distribution. Before entering a competitive match, players are required to stake a defined amount of cryptocurrency, ensuring commitment and fairness. At the end of each match, the smart contract automatically distributes rewards to the winner in a secure and transparent manner. These blockchain features help maintain trust and prevent manipulation of match outcomes.

3.2.4 Multiplayer Gameplay

Multiplayer gameplay is supported through Mirror Networking, offering real-time movement synchronization across all connected players. The system ensures that game states, player positions, and elimination decisions remain consistent for everyone in the session. This creates a fair and reliable multiplayer environment where actions and results are uniformly updated on all devices.

3.2.5 Single Player Mode

The single-player mode allows users to practice without the pressure of competing against real players. It helps users understand controls, movement timing, and overall game flow. This mode also provides an accessible entry point for new players who want to get comfortable before switching to multiplayer matches.

3.2.6 Multiplayer Mode

The multiplayer mode enables players to compete with real opponents, where cryptocurrency bets are placed and the final winner receives the prize. This mode is designed to deliver a highly

competitive environment supported by blockchain-backed transparency. Real-time interactions and synchronized states ensure that the match outcome is based purely on player skill and reaction time.

3.2.7 In-App purchases

The system also includes in-app purchases, allowing players to buy coins [10] that can be used for matches or to unlock additional game-related features. This provides an alternative to cryptocurrency staking and helps maintain the game's economic balance. A secure payment flow ensures user trust and smooth purchasing experience.

3.2.8 Manage Users

Finally, administrators can manage users, adjust bet amounts, edit in-game asset prices, and control the number of players allowed in each session. This administrative functionality ensures flexibility and smooth operation of the platform. It also allows the developers to maintain game fairness, adjust difficulty, and handle community management effectively.

3.3 Nonfunctional Requirements

Nonfunctional requirements define the quality attributes, constraints, and performance standards that determine how the Red-Light Green-Light game operates. These requirements ensure reliability, efficiency, and a smooth user experience throughout the system.

3.3.1 Performance

Performance requirements focus on delivering smooth, low-latency gameplay regardless of the number of players in a session. The system must ensure that player movements, game states, and elimination events update instantly without lag. Azure Functions and optimized backend processes help maintain fast response times and stable real-time interactions.

3.3.2 Security

Security is a key requirement for this system, especially because it handles cryptocurrency transactions and sensitive user data. Player information must be encrypted [12] to prevent unauthorized access, and the smart contracts must be implemented securely to avoid exploits or vulnerabilities. These measures ensure that both gameplay and blockchain interactions remain safe for all users.

3.3.3 Scalability

Scalability is essential because the game may have many players joining simultaneously. The system must be capable of handling multiple concurrent users without performance drops or synchronization issues. Efficient handling of blockchain [14] transactions and backend requests ensures that the platform can expand as the user base grows.

3.3.4 Ease of Use

Ease of use is also a priority, ensuring that the game remains accessible to players of all skill levels. The interface should be simple, intuitive, and easy to navigate, allowing users to join games, stake coins, connect their wallets, and purchase in-game assets without confusion. A smooth user experience contributes to player satisfaction and long-term engagement.

Chapter 4

Design and Architecture

4 Design and Architecture

4.1 System Architecture

The structure of the system explains how Red-Light Green-Light operates by describing its core components, how they are organized, and the way they interact with each other to ensure smooth functionality throughout the platform.

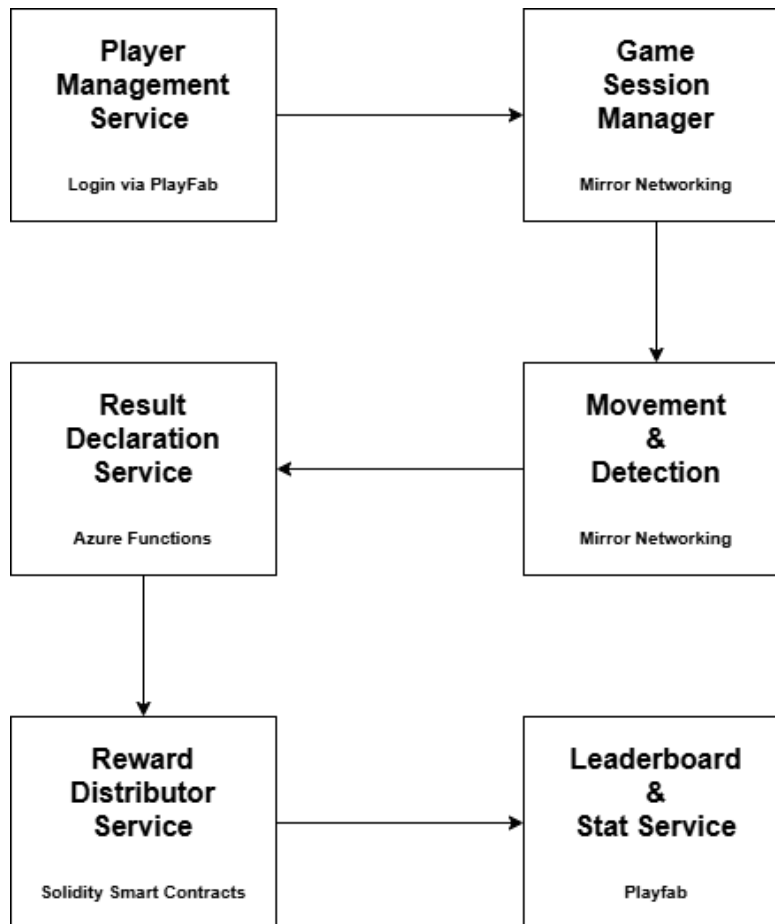


Figure 4.1 Red-Light Green-Light Architecture

The system is built as a combination of interconnected services, where each component performs a specific task to support multiplayer gameplay, authentication, backend processing, and blockchain-based reward distribution. At the entry point of the system, the Player Management Service handles user login and authentication using PlayFab. Once the player is authenticated, the Game Session Manager uses Mirror Networking to create or join multiplayer sessions, synchronize game states, and manage communication between all connected players.

Within the gameplay environment, the Movement and Detection module also uses Mirror Networking to track player movement in real time. This module evaluates whether players move during the Red Light phase and triggers elimination when necessary. As the match progresses, once a winner is determined, the Result Declaration Service communicates with Azure Functions. The Azure backend validates the match result and ensures that the correct winner information is forwarded to the blockchain.

The Reward Distributor Service uses a Solidity smart contract to store and manage reward distribution. When Azure Functions confirm the game outcome, the smart contract records the winner and distributes rewards securely on the blockchain. Finally, the Leaderboard and Statistics Service uses PlayFab to update and display player stats, match history, and leaderboard rankings. All components work together to ensure smooth gameplay, secure result handling, and transparent reward distribution.

4.2 Data Representation

The system manages several important data entities that support gameplay, user tracking, and blockchain operations. The Player entity includes fields such as `playerId`, `username`, `walletAddress`, `authenticationToken`, and `currentStatistics` like wins, losses, and coins. This information is stored and managed through PlayFab. The Game Session entity stores `matchId`, `sessionPlayers`, `startTime`, `currentState`, and `winnerId`. It also tracks phase changes such as Red Light or Green Light and maintains synchronized game data for all connected clients.

Movement data is represented as positional coordinates, velocity, and timestamps for each player. This helps the Movement and Detection module determine whether a player has moved during restricted phases. Result data includes `matchId`, `winnerAddress`, `verificationStatus`, and `timestamp`, which are passed to Azure Functions and then stored in the blockchain through the smart contract. Reward data contains `rewardAmount`, `playerWallet`, `transactionHash`, and `contractStatus`, ensuring transparent prize distribution.

The leaderboard data structure contains `playerId`, `totalWins`, `accuracyScore`, and `fastestCompletionTime`, all maintained by PlayFab's backend services. These data entities work together to support authentication, gameplay logic, match verification, blockchain storage, and player ranking within the system.

4.3 Process Flow

The system begins when the player opens the game and logs in through the Player Management Service using PlayFab authentication. After verification, the user can either enter single-player mode or join a multiplayer match. In multiplayer mode, the Game Session Manager uses Mirror Networking to place the user in a game lobby and synchronize their connection with other players. Once the session is ready, the game transitions into the Red-Light Green-Light sequence. During the match, the Movement and Detection module continuously tracks player movement and ensures that all actions are synchronized across clients. When the game switches to the Red Light phase, the system monitors each player to detect illegal movement and eliminates players who fail to stop in time. When a winner is declared, the Result Declaration Service sends match information to Azure Functions, where backend logic verifies the outcome. After verification, the backend sends the winner data to the blockchain through the Reward Distributor Service. The Solidity smart contract records the winner and distributes the reward automatically. Finally, the Leaderboard and Stat Service updates PlayFab with new player stats, total wins, and leaderboard rankings. The process ends when the system returns the results to the players.

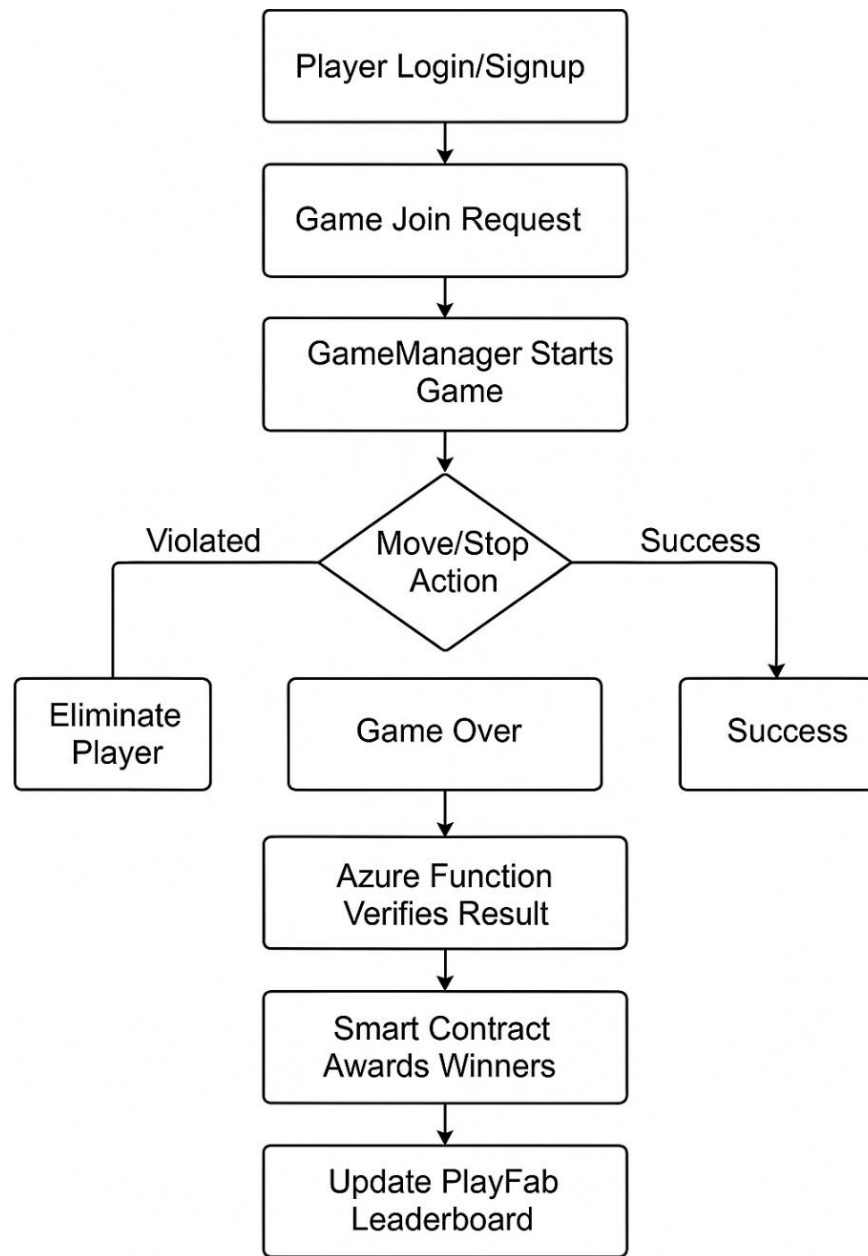


Figure 4.2 Red-Light Green-Light Process Flow

4.4 Sequence Diagram

The sequence diagram provides a flow in which the player first logs into the game using PlayFab. After joining a multiplayer session, the GameManager starts the game and the DollController manages the red/green light phases. Player actions are validated, and results are verified via Azure Functions, rewarded through smart contracts, and updated on PlayFab. The following is the Sequence Diagram of our project.

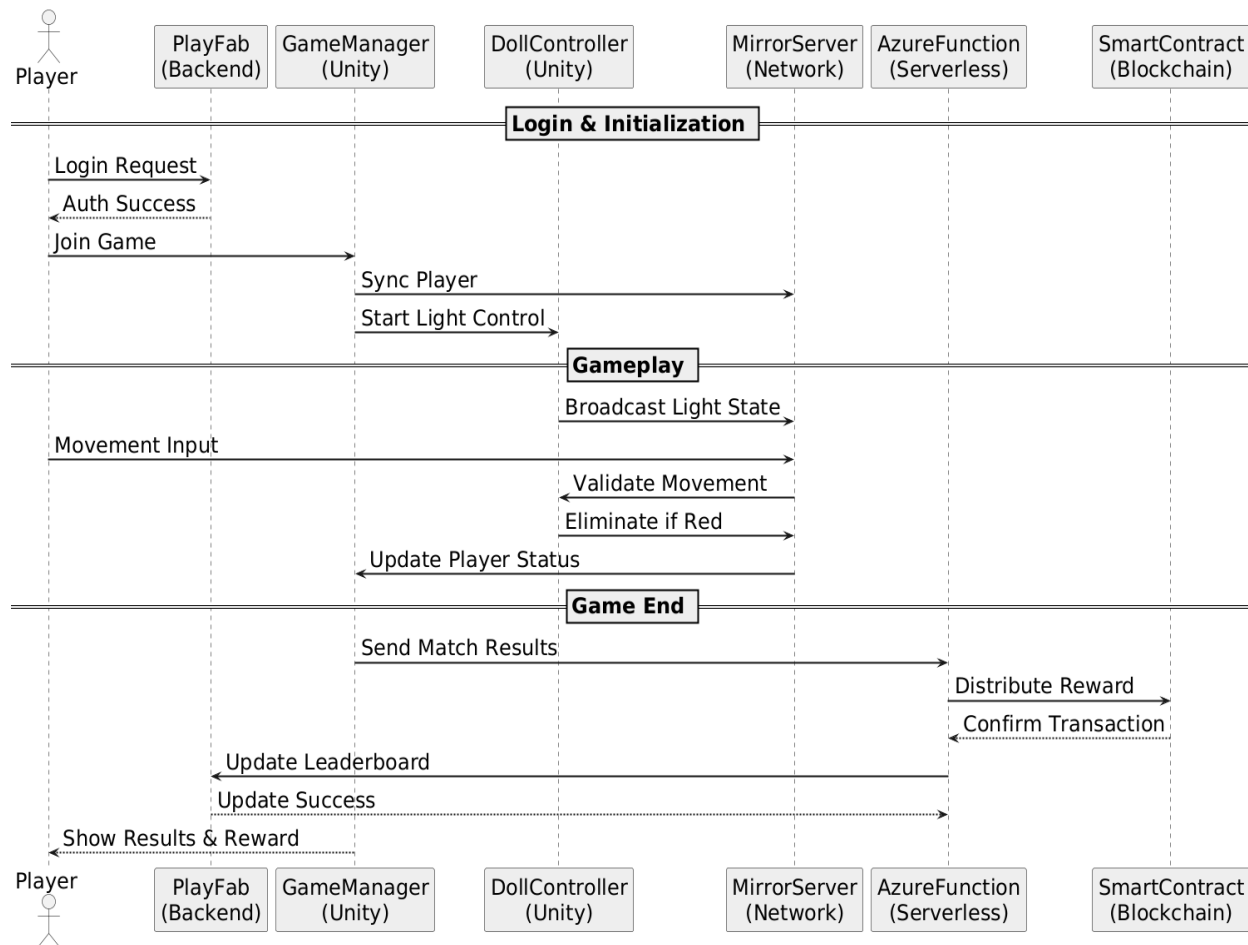


Figure 4.3 Red-Light Green-Light Sequence Diagram

The sequence begins with the player logging in through PlayFab, which verifies credentials and returns authentication status. Once authenticated, the player communicates with the Game Session Manager to join or create a multiplayer session. Mirror Networking then synchronizes all connected players and ensures that each client receives updated positions, game states, and timing information.

During gameplay, the Movement and Detection module takes continuous input from players and transmits it to the server through Mirror. The server checks movement validity, especially during the Red Light phase, and broadcasts elimination messages when necessary. After a winner is identified, the server sends the winner data to Azure Functions for verification. Azure confirms the match result and triggers the smart contract to store the final outcome on the blockchain.

The blockchain responds with a transaction hash, which verifies that the result is stored securely. Azure then updates PlayFab with the final match details, including rewards and leaderboard

updates. The game clients receive the final match results and display the winner, earnings, and updated stats. The sequence ends when all players exit the session or return to the main menu.

4.5 Class Diagram

The class structure of the system includes several core classes, each representing a major part of the gameplay and backend logic. The Player Controller class manages player input, position updates, and communication with the networking layer. The GameManager class controls match flow, timing for Red Light and Green Light phases, and elimination rules. The Network Manager class, provided by Mirror Networking, handles synchronization, player spawning, and message transmission between clients and the server.

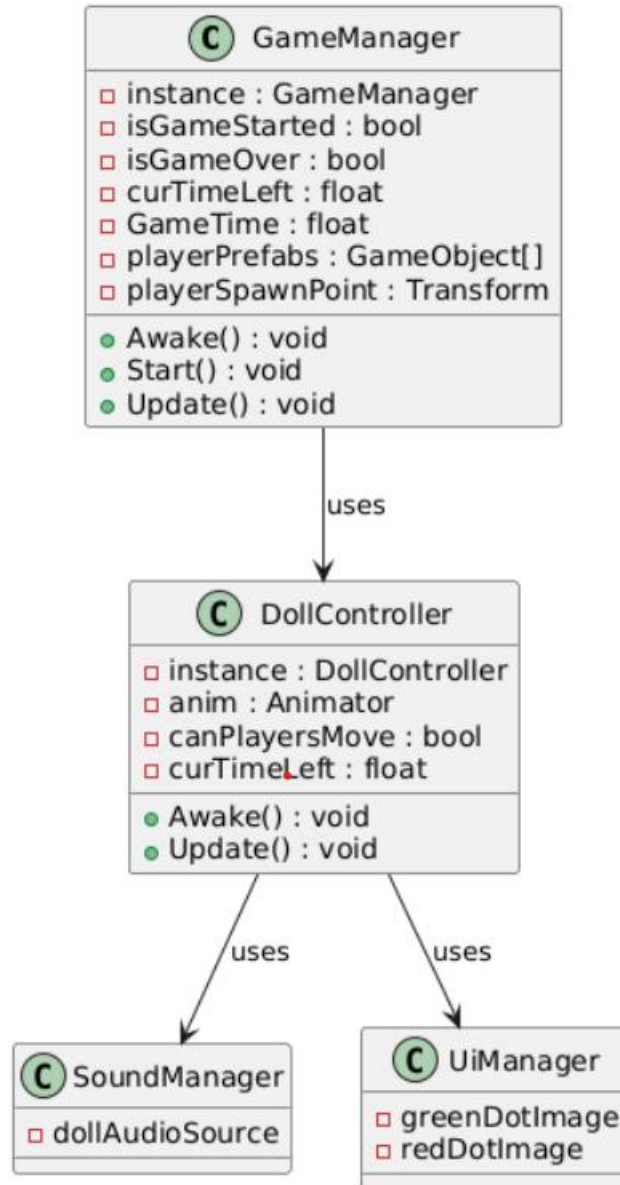


Figure 4.4 Red-Light Green-Light Class Diagram

4.5.1 Components

The components are as follows:

4.5.1.1 GameManager:

The GameManager acts as the main controller of the game, handling essential functions such as starting the game, ending it, and managing the countdown timer. It also manages player spawning by storing player prefabs and the spawn point, and through its Awake, Start, and Update methods, it keeps the overall game flow running. The GameManager makes use of the DollController to manage time-based game logic and control when players are allowed or not allowed to move.

4.5.1.2 DollController:

The DollController manages the doll's behaviour, animations, and rules related to player movement. It keeps track of whether players are allowed to move, the animation state, and the current time left for switching between movement phases. Its Awake and Update methods control the doll's actions during gameplay. It depends on the SoundManager to play audio cues and the UIManager to update visual indicators such as red or green signals for the players.

4.5.1.3 SoundManager:

The SoundManager is responsible for all audio-related tasks in the game, particularly playing the doll's sound effects when triggered by the DollController. It stores the necessary audio sources and ensures that the correct sounds play at the right moments, keeping audio management organized and separate from gameplay logic.

4.5.1.4 UIManager:

The UIManager handles the visual user interface elements, especially the indicators that show whether players can move or must stop. It stores references to the red and green dot images and updates them whenever the DollController changes the game state. This ensures that players receive clear visual cues throughout the game.

4.5.2 Relationships

The relationships are as follows:

4.5.2.1 GameManager and DollController:

The GameManager controls the overall game flow and uses the DollController to manage gameplay states such as when players can move or must stop.

4.5.2.2 GameManager and SoundManager / UIManager (Indirect):

The GameManager does not directly use SoundManager or UIManager but influences them through the DollController, which handles sound playback and UI updates based on game state.

4.5.2.3 DollController and SoundManager:

The DollController uses the SoundManager to play doll-related sounds, such as alerts or movement cues during the game sequence.

4.5.2.4 DollController and UIManager:

The DollController updates the UIManager to change on-screen indicators (e.g., red and green signals) that inform players whether movement is allowed.

4.5.2.5 GameManager and Player Objects:

The GameManager maintains player prefabs and spawn points, controlling when and where players appear in the game.

4.5.2.6 DollController and Animation System:

The DollController interacts with the animation component to switch between animation states depending on the game's logic (e.g., facing forward or backward).

Chapter 5

Implementation

5 Implementation

In the implementation phase of the Red-Light Green-Light game, we developed core algorithms that ensure smooth, intelligent, and efficient system performance, enabling players to interact seamlessly.

5.1 Algorithm

The core intelligence of the Red-Light Green-Light game is powered by a set of high-performance algorithms implemented in unity using C#. Each algorithm is optimized for accuracy, efficiency, and scalability, ensuring smooth real-time performance for players.

5.1.1 LightSwitch Algorithm

The LightSwitch algorithm controls the core game mechanic by switching between Green Light and Red Light states at predefined or random intervals. When the timer reaches a threshold, the algorithm changes the state and broadcasts it to all connected players through Mirror Networking. During Green Light, players are allowed to move freely, while Red Light activates movement monitoring. This algorithm ensures synchronized and fair phase transitions across all clients in the multiplayer session.

5.1.2 Movement Detection Algorithm

The Movement Detection algorithm monitors each player's position in real time and compares their current location with the last recorded position. If the system detects any movement during the Red Light phase, it flags the player as violating the game rules. This algorithm ensures high accuracy by using continuous position tracking, timestamps, and synchronized state data from Mirror Networking. It is optimized for smooth real-time detection even under network latency.

5.1.3 Elimination Algorithm

The Elimination algorithm is triggered when a violation is detected during the Red Light phase. Once a player is marked for movement, the algorithm changes their status to eliminated and notifies all clients about their removal. This update includes disabling player movement, updating the camera or UI, and sending elimination data to the server. The algorithm ensures fairness by executing elimination on the server side, preventing cheating or manipulation.

5.1.4 Mirror Networking

Mirror Networking handles real-time multiplayer communication between players and the server. The algorithm synchronizes player positions, game states, elimination events, and scene transitions. It also manages player joining, leaving, message relays, and authority checks. Mirror ensures that all players receive consistent updates, allowing smooth and fair gameplay regardless of network conditions.

5.1.5 Azure Functions

Azure Functions provide serverless backend logic to handle match validation, result verification, and communication with the blockchain. The algorithm receives match data from the Unity client, checks winner information, and triggers the smart contract for reward distribution. It operates without needing a dedicated server and scales automatically, ensuring fast response and secure backend processing.

5.1.6 Smart Contract (Solidity)

The Solidity algorithm records match results on the blockchain and manages reward payouts. It receives winner information from Azure Functions and updates the on-chain data structure. The algorithm ensures transparency by making the match outcome viewable and immutable. It also transfers cryptocurrency rewards securely to the winner's wallet, removing the possibility of tampering or unfair payouts.

5.1.7 PlayFab Integration

The PlayFab integration algorithm manages user authentication, player profiles, stats, and leaderboard data. It verifies login credentials, stores match results, and updates player progress automatically. The system ensures secure cloud storage for all player-related information and supports features like currency management, leaderboards, and analytics.

5.2 User Interface

The user interface (UI) of Red-Light Green-Light is designed to be simple, intuitive, and visually appealing. Below are the main screens:

Figure 5.1: Home Page – Displays the four buttons for play, characters, leaderboards and withdraw.



Figure 5.1 Red-Light Green-Light Home Page

The screenshot showcases the main menu of a game titled “**Red Light Green Light,**” presented with a bold, stylized logo at the top center featuring red, green, and blue geometric accents that resemble traffic-light signals. The background displays a brightly lit, playful yet tense game arena with cartoon-like wall decorations clouds, shapes, and pastel colors—contrasted by the presence of masked guards dressed in red jumpsuits and positioned on both sides, hinting at the high-stakes nature of the game environment. Centered on the screen is a vertical stack of large, vibrant menu buttons, each designed with rounded edges and high-contrast colors for instant visibility. The **Play** button appears at the top in bright yellow, followed by **Characters** in red, **Leaderboard** in green, and **Withdraw** in deep red guiding players intuitively through the game’s core options. In the upper-right corner, a prominent coin display shows the player’s in-game currency, visually framed by a gold coin icon and a plus sign for easy access to earning or purchasing more. A speaker icon in the upper-left corner offers simple audio control.

Figure 5.2: Gameplay – Shows detailed environment which contains game arena, doll and character movement in game.



Figure 5.2 Gameplay

The screen showcases the environment includes a simple game arena with a starting line, finish line, and a doll positioned at the front to control Red and Green Light phases. Players move their characters across the field during Green Light, and the doll monitors them during Red Light. The setup provides clear visibility of the doll, player movements, and the path they must follow to win.

Figure 5.3: Lobby – Displays the waiting area in which a player waits for other players to join before entering the game arena.

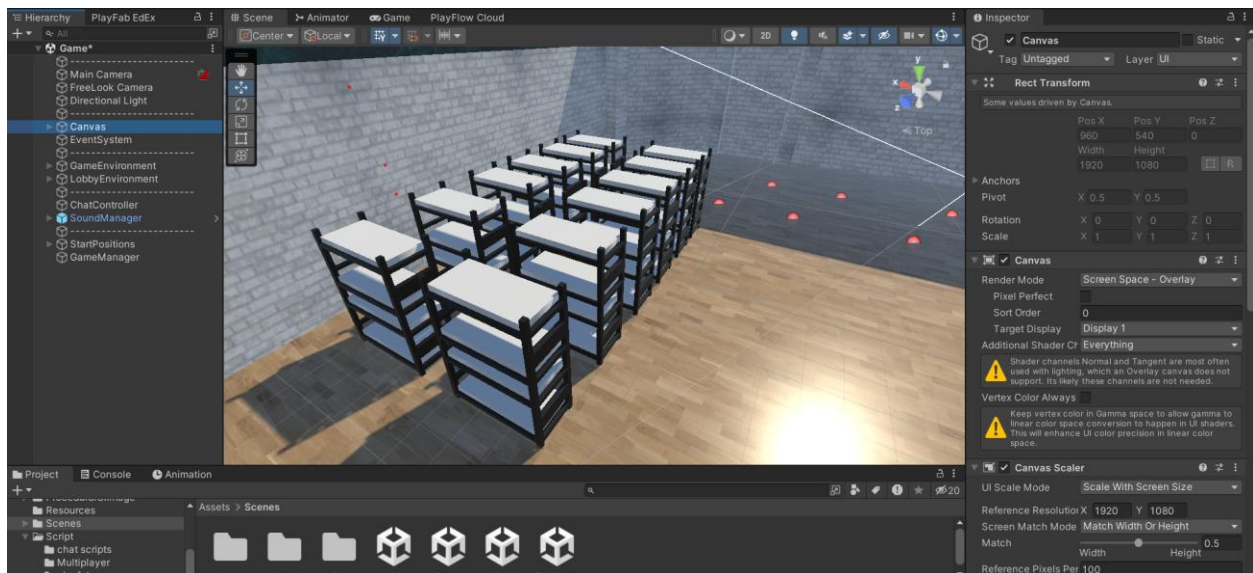


Figure 5.3 Waiting Lobby

The screen showcases the list of players who have joined the session and updates in real time as new players enter. The lobby also displays the room code, match settings, and the total number of required players. A countdown timer starts once enough players have joined. It includes player name cards, a status message showing “Waiting for players...”, and a ready indicator for each player. There is also a leave button, a start button for the host, and a simple background that matches the game theme. The lobby helps players prepare, see who is joining, and wait comfortably before entering the main game arena.

Figure 5.4: PlayFab –Stores information about users like username, password and leaderboard.

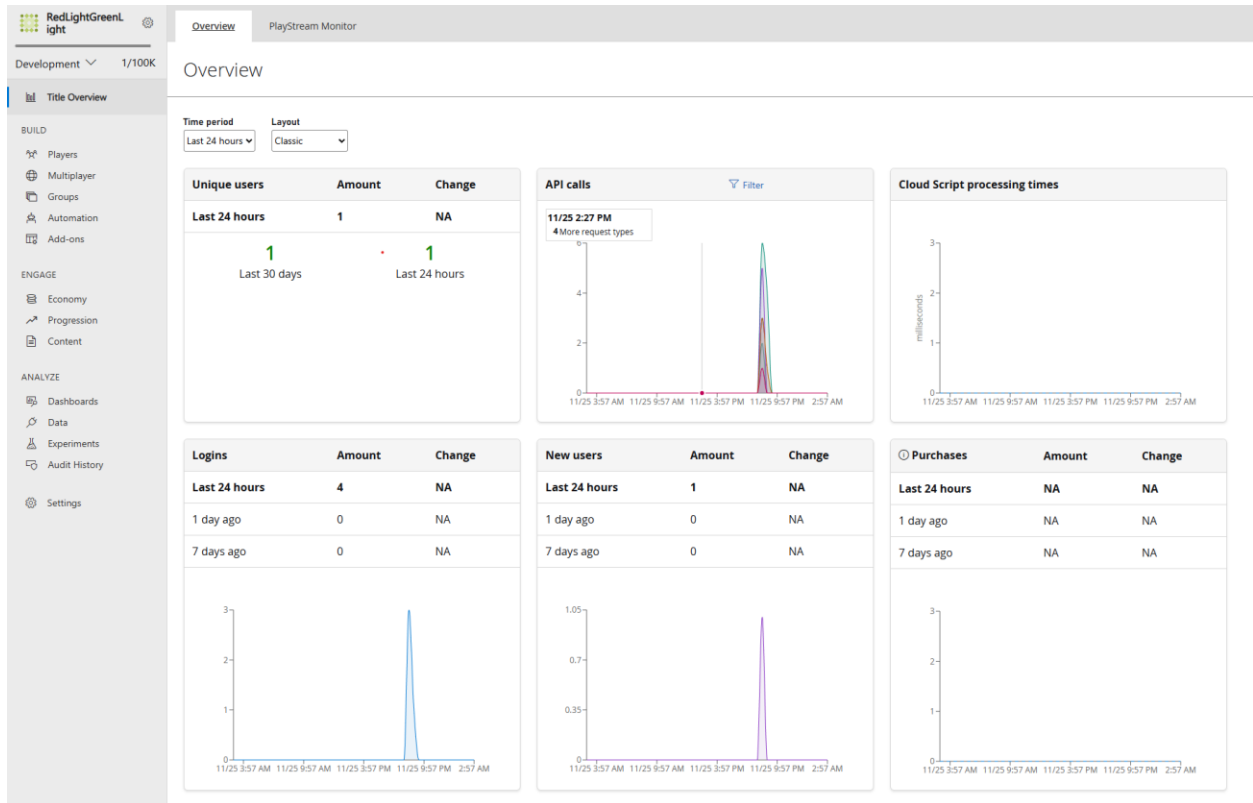


Figure 5.4 PlayFab Studio

The screen showcases PlayFab Studio that manages user data including usernames, passwords, and player profiles. It also handles leaderboards, player statistics, and game-related records. Additionally, it supports authentication, data storage, and cloud features that help keep player information organized and secure.

Figure 5.5: Withdraw –User will withdraw coins to his crypto wallet.



Figure 5.5 Withdraw Panel

The screen showcases Withdraw Panel where users can transfer their in-game coins to their personal crypto wallet. It shows the user's current coin balance and allows them to enter the amount they wish to withdraw along with their wallet address. The panel validates the input to ensure the withdrawal amount is within the available balance and that the wallet address is correctly formatted. It includes a balance display, input fields for the withdrawal amount and wallet address, a withdraw button, and a notification area that shows success or error messages. There is also a close button to exit the panel. The design is clean, simple, and matches the theme of the game, providing a straightforward and comfortable experience for users managing their coins.

Chapter 6

Testing and Evaluation

6 Testing and Evaluation

Testing and evaluation for the Red-Light-Green Light multiplayer game focused on ensuring that all major systems, including networking, movement detection, blockchain integration, and backend services, functioned reliably under real gameplay conditions. The testing process began with unit testing of individual scripts in Unity, such as player movement, light-switch timing, elimination logic, and collision detection. Each script was tested separately to confirm that core mechanics like state transitions and player control behaved as expected. Early unit tests helped identify issues such as delayed movement detection and synchronization gaps between clients.

6.1 Manual Testing

Manual testing was performed throughout the development of the Red Light Green Light multiplayer game to ensure that all features behaved as expected and that the user experience remained smooth and error-free. The process began with testing the basic gameplay mechanics, where testers played multiple rounds to verify the movement controls, accuracy of Red Light and Green Light transitions, and responsiveness of character animations. Each phase transition was observed to confirm that players could move only during the allowed state and that movement restrictions activated correctly. Manual testing was also applied to backend and Web3 features. Testers logged in with different PlayFab accounts to verify user authentication, profile retrieval, and leaderboard updates. They manually triggered match endings to observe how the Azure Function responded and whether the correct winner information was forwarded to the blockchain.

6.1.1 System Testing

System testing was carried out to evaluate the complete end-to-end functionality of the Red Light Green Light multiplayer game and ensure that all integrated components worked together as a unified system. This phase focused on testing the entire workflow, starting from user authentication and matchmaking to movement detection, elimination logic, backend verification, and final reward distribution. The purpose was to validate that the overall system performed correctly under realistic conditions and that there were no failures during full game sessions.

The system was tested by running complete multiplayer matches with multiple clients connected simultaneously. These tests verified Mirror Networking synchronization, ensuring that player

positions, game states, and elimination updates were consistent across all devices. The system was observed for issues such as delayed updates, inconsistent light switching, and mismatched elimination events. Any irregularities were documented, reproduced, and fixed to ensure stable and fair gameplay.

6.1.2 Unit Testing

Unit testing focused on verifying the functionality of individual components within the Red Light Green Light game before integrating them into the larger system. Each script, function, and algorithm was tested separately to ensure it performed its specific task correctly. The goal was to identify and eliminate errors early in development so they would not affect other parts of the project during later stages. The LightSwitch algorithm was tested by repeatedly triggering Red and Green Light transitions to confirm that timing intervals, state changes, and broadcast commands worked as intended. Movement detection was isolated and tested using controlled player inputs to ensure the algorithm correctly identified both valid and invalid movements under different conditions. Elimination logic was also tested independently by simulating violations and checking whether the system marked players as eliminated and updated their status accurately.

Unit Testing: Authentication & User Management

Authentication is a core function of Red-Light Green-Light, ensuring secure access for all users. The following unit tests verify the login, signup. These tests help confirm that valid credentials allow access while invalid inputs are handled safely.

Table 6.1 Unit Testing: Authentication & User Management

Test Case ID	Test Description	Input	Expected Output	Remarks
AUTH-01	Login with valid credentials	Email + Correct Password	Login success, dashboard displayed	Pass
AUTH-02	Login with invalid password	Email + Wrong Password	“Invalid credentials” error	Pass
AUTH-03	Signup with missing fields	Email only	Field validation errors	Pass

6.1.3 Functional Testing

Functional testing in Red-Light Green-Light is carried out after unit testing to ensure that every module behaves exactly as defined in the system requirements. All major player roles are tested to confirm that their respective features load correctly, display accurate data, and respond as expected to user actions.

Functional Testing 1: Login (All User Roles)

This test validates that all Red-Light Green-Light users can access the system securely and are redirected to the correct dashboard.

Table 6.2 Functional Testing

Test Case ID	Test Case Name	Attributes & Values Tested	Expected Result	Actual Result
FT-001	Login with valid player credentials	Username: player1@example.com Password: *****	Player successfully logs in and enters the Waiting Lobby	Pass
FT-002	Login with invalid credentials	Wrong username or wrong password	System displays “Invalid username or password”	Pass
FT-003	Login with network failure	Correct credentials but server offline	Error message displayed: “Unable to connect to server”	Pass
FT-004	Logout	User clicks logout button	User returns to login screen and session ends	Pass

The functional testing table for the login process evaluates the behavior of the Kaswa Scholar authentication system across all three user categories—Students, Recruiters, and Admins [17]. Test Case FT-001 verifies the student login flow by checking whether valid credentials correctly redirect the student to a personalized dashboard displaying their profile summary and recommended opportunities. Following this, Test Case FT-002 ensures that recruiters logging in with valid details are accurately directed to their designated dashboard, which includes analytics, saved searches, and talent insights. Test Case FT-003 confirms that administrators gain access to the system overview dashboard upon entering correct login information. Test Case FT-004 examines the system’s ability to detect invalid login attempts by providing an incorrect email or password; the expected “Invalid credentials” message is displayed, indicating proper error

handling. Test Case FT-005 ensures robust validation by testing empty fields, where the system appropriately prompts the user that all fields are required. Finally, Test Case FT-006 confirms that the logout feature terminates the user session securely and redirects them back to the login interface. All test cases passed successfully, demonstrating that Kaswa Scholar's authentication module consistently delivers secure access control and accurate dashboard routing for all user types.

6.1.4 Integration Testing

Integration testing was conducted to verify that all interconnected modules of the Red-Light Green-Light multiplayer game interact smoothly and perform as expected when combined. This phase ensured proper communication between major components such as the login system, PlayFab backend, lobby manager, multiplayer networking, game arena logic, doll light controller, movement detection system, elimination module, and blockchain reward services. The testing validated that players can seamlessly move from login to the lobby, transition into the gameplay arena, experience synchronized Red Light/Green Light states, and have their eliminations and movements correctly updated across all clients. Additionally, blockchain functions such as reward distribution and withdrawal requests were checked to ensure secure and successful interactions between the game and wallet APIs. All integrated modules performed reliably, confirming that the overall system works cohesively in real gameplay scenarios.

Integration Testing 1: Login as Admin

Table 6.3 Integration Testing

Test Case ID	Test Case Name	Attributes & Modules Involved	Expected Result	Actual Result
IT-001	Login → Lobby Integration	Login Module + PlayFab Authentication + Lobby Manager	After successful login, player is automatically redirected to Waiting Lobby with correct player name and ID	Pass

IT-002	Lobby → Game Arena Transition	Lobby Manager + Multiplayer Matchmaker + Scene Loader	When minimum players join, game session starts and all players move to Game Arena Scene together	Pass
IT-003	Game Arena → Doll Light System	Arena Scene + Doll Controller + Light State Manager	Doll correctly switches between Red Light / Green Light states and UI updates according	Pass
IT-004	Movement Detection → Elimination System	Player Movement Script + Red Light Detector	When player moves during Red Light, elimination triggers and syncs across all clients	Pass
IT-005	Multiplayer Sync → Server State	Mirror Networking + Game Manager	Player positions, eliminations, and timer values remain synchronized across all connected players	Pass

6.2 Tools and Technologies

Table 6.4 Tools and Technologies

	Tools	Version	Rationale
	MS Word	2019	Documentation and report writing
	MS PowerPoint	2019	Presentation creation for project defense
	Adobe Illustrator	2021	UI design and graphics creation

Tools And Technologies	Figma	Latest	UI/UX design and prototyping
	Postman	Latest	API testing and integration
	Git & GitHub	Latest	Version control and collaborative development
	MS Project	2019	Gant Chart
	Unity	2022	Game Development Engine
	Adobe Photoshop	2015	Background design and banner art

6.3 Summary

This project is a multiplayer Red Light, Green Light game built using Unity with PlayFab for backend services, Azure Functions for server-side logic, and Solidity smart contracts for blockchain-based coin management. The game allows multiple players to join a waiting lobby, where their progress and rankings are tracked in real time. Players navigate a game arena under the control of a doll that alternates between “Red Light” and “Green Light,” with movement detection determining elimination. The system integrates crypto rewards, allowing players to withdraw coins securely to their wallets. The project emphasizes responsive UI design, smooth character movement, reliable networking, and secure backend management, providing an interactive and engaging gaming experience while demonstrating a combination of game development, multiplayer networking, and blockchain integration.

Chapter 7

Conclusion and Future Work

7 Conclusion and Future Work

7.1 Conclusion

This project successfully demonstrated how traditional multiplayer game development can be enhanced through the integration of modern cloud technologies and decentralized systems. By building a multiplayer version of the Red-Light Green-Light game, the project explored how real-time gameplay, fairness, and trust can be improved by combining Unity, Mirror Networking, PlayFab, Azure Functions, and blockchain smart contracts. The final system delivers a complete and functional experience where players can compete, interact, and view results that are transparent and tamper-proof.

The use of blockchain technology added an important layer of trust to the game by ensuring that match results and reward distribution are recorded securely. Smart contracts were used to manage staking and reward payouts, making the system more transparent compared to traditional centralized game servers. This approach shows how decentralized mechanisms can be used to support competitive online games, especially in scenarios where fairness and verifiable outcomes are important.

Cloud integration through PlayFab and Azure Functions played a key role in managing player authentication, backend processing, and leaderboard updates. These services allowed the system to remain scalable, secure, and easy to maintain without relying on a dedicated backend server. Through serverless functions, the game was able to communicate with the blockchain securely and efficiently, demonstrating how cloud and Web3 technologies can complement each other.

The project also highlighted the importance of real-time synchronization and movement detection, which were handled through Mirror Networking. This ensured that all players experienced consistent gameplay, proper elimination logic, and smooth updates across all devices. Developing and testing this component reinforced the significance of networking accuracy and delay management in multiplayer games.

Overall, the project achieved its goal of creating a modern, competitive game that merges real-time multiplayer functionality with decentralized result verification. It provides a strong foundation for future development, such as expanding to mobile platforms, adding new game modes, or introducing more advanced Web3 features. The work completed in this project

demonstrates how emerging technologies can be integrated into interactive entertainment to provide new levels of fairness, transparency, and player engagement [16].

7.2 Future Work

While the current version of Red-Light Green-Light is fully functional and meets its objectives, there is significant potential to enhance its features, usability, and reach. Future work will focus on making the platform even more intelligent, accessible, and user-friendly:

7.2.1 Mobile Applications (Android & iOS)

The game can be launched for both Android and iOS. You just switch platform and build it for Android and iOS. This will greatly increase number of users and help to scale the game at even higher level [12].

7.2.2 Advanced AI Features

We can also introduce a face detection system in game which will make the game more interesting and harder. Camera access can be gained through unity and face detection can be added through OpenCVForUnity or Using Unity Barracuda + MediaPipe (Google's ML model).

7.2.3 Enhanced Character Customization

We can also introduce character customization like clothes, hats, glasses etc. which can be bought through in-app coins, or we can also add NFTs. This way user will feel more involved in our game and it will target other audiences as well.

7.2.4 Strong Security and Privacy Measures

Implement robust security protocols, including two-factor authentication (2FA), end-to-end encryption, and strict privacy controls to protect sensitive data [17].

7.2.5 Performance and Scalability Optimization

Optimize the game's speed, loading time, and responsiveness to support an increasing number of users without any lag or downtime.

8 References

- [1] M. Dezhkam, S. Xue, and Z. Liu, “Project Management System, Concepts and Approach Foundations,” *IOP Conference Series: Earth and Environmental Science*, vol. 310, no. 5, 2019. [Online]. Available: <https://iopscience.iop.org/article/10.1088/1755-1315/310/5/052081>
- [2] World Economic Forum, “Building Block(chain)s for a Better Planet: Sustainable Development Opportunities with Blockchain,” World Economic Forum, 2019., Accessed on: March 26, 2023.
- [3] B. M. Ramageri et M. Arjunwadkar, “Applications of Blockchain Technology in Various Sectors:A Review,” *International Journal of Future Generation Communication and Networking* , vol. 13, n° %12, pp. 94-99, April 2020.
- [4] Unity Documentation: <https://docs.unity3d.com/>
- [5] “Blockchain for Social Impact: Moving Beyond the Hype,” [En ligne]. Available: <https://www.rippleworks.org/blockchain/>.
- [6] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” *arXiv*, Oct. 2019. [Online]. Available: <https://arxiv.org/abs/1810.04805>
- [7] PlayFab API Reference: <https://learn.microsoft.com/en-us/gaming/playfab/>
- [8] E. Stevens, “The Complete Guide to UI/UX Design,” AND Academy, Jan. 16, 2023. [Online]. Available: <https://www.andacademy.com/resources/blog/ui-ux-design/what-is-ui-ux-design-the-complete-guide/>
- [9] UX Design Institute, “A Complete Guide to the UI Design Process,” Oct. 12, 2023. [Online]. Available: <https://www.uxdesigninstitute.com/blog/guide-to-the-ui-design-process/>
- [10] Ethereum Smart Contracts Guide: <https://ethereum.org/en/developers/docs/smart-contracts/>
- [11] D. Vohra, “Implementing JSON Web Tokens (JWT) for Secure Authentication,” *International Journal of Computer Applications*, vol. 179, no. 23, pp. 8–12, 2018. doi: 10.5120/ijca2018917206

- [12] D. Fett, R. Küsters, and G. Schmitz, “A Comprehensive Formal Security Analysis of OAuth 2.0,” in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 2016, pp. 1204–1215. doi: 10.1145/2976749.2978325
- [13] Web3.js Documentation: <https://web3js.readthedocs.io/>
- [14] N. S. Sirisha, T. Agarwal, R. Monde, R. Yadav et R. Hande, “Proposed Solution for Trackable Donations using Blockchain,” chez 2019 International Conference on Nascent Technologies in Engineering (ICNTE), Navi Mumbai, India, 02 January 2020.
- [15] https://www.tutorialspoint.com/sdlc/sdlc_iterative_model.htm